

Отлично! Ты хочешь получить полноценный **Технический проект (ТЗ)** уровня **senior-разработчика**. Сделаем по-взрослому: с архитектурой, обоснованием решений, детальной структурой и комментариями для каждого файла.

ТЕХНИЧЕСКИЙ ПРОЕКТ

Chelbo — Кроссплатформенный мессенджер

Дипломная работа (Академия TOP)

1. ОБЩАЯ ИНФОРМАЦИЯ

Параметр	Значение
Название проекта	Chelbo
Тип	Real-time кроссплатформенный мессенджер
Архитектура	Клиент-серверная (WebSocket + REST API)
Целевая аудитория	Пользователи РФ, ищущие замену Telegram
Платформы	Web (основная), PWA, в перспективе мобильные приложения

2. БИЗНЕС-ТРЕБОВАНИЯ

2.1 Проблема

- Telegram замедлен и под санкционными рисками
- WhatsApp принадлежит Meta (экстремистская организация в РФ)
- VK Мессенджер имеет ограничения по приватности
- Нет российского мессенджера с современным API и Web-first подходом

2.2 Решение

Chelbo — российский мессенджер с открытой архитектурой, работающий через Web и PWA, без необходимости установки приложения.

2.3 Ключевые метрики (цели на 6 месяцев)

Метрика	Целевое значение
DAU (дневная аудитория)	10 000
MAU (месячная аудитория)	50 000
Сообщений в день	1 000 000
Среднее время ответа API	< 50 мс
Uptime	99.9%

3. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

3.1 Модуль авторизации (P0 - критично)

ID	Требование	Приоритет
AUTH-01	Регистрация по email + пароль	P0
AUTH-02	Вход по email + пароль	P0
AUTH-03	Выход из аккаунта	P0
AUTH-04	Восстановление пароля (email со ссылкой)	P1
AUTH-05	Изменение профиля (имя, аватар, bio)	P1
AUTH-06	Удаление аккаунта	P2

3.2 Модуль личных сообщений (P0)

ID	Требование	Приоритет
----	------------	-----------

CHAT-01	Отправка текстовых сообщений в real-time	P0
CHAT-02	Получение сообщений через WebSocket	P0
CHAT-03	История сообщений (пагинация по 50)	P0
CHAT-04	Статус "доставлено" (сервер подтверждает)	P0
CHAT-05	Статус "прочитано"	P1
CHAT-06	Уведомление "пользователь печатает"	P2

3.3 Модуль групповых чатов (P1)

ID	Требование	Приоритет
GROUP-01	Создание группы (название, аватар)	P1
GROUP-02	Добавление участников по email/имени	P1
GROUP-03	Выход из группы	P1
GROUP-04	Назначение администраторов	P2

3.4 Модуль файлов (P1)

ID	Требование	Приоритет
FILE-01	Отправка изображений (до 10 MB)	P1
FILE-02	Отправка документов (до 50 MB)	P1
FILE-03	Превью изображений	P1
FILE-04	Авто-сжатие больших изображений	P2

3.5 Модуль AI-ассистента (P1 - фишка для диплома)

ID	Требование	Приоритет
AI-01	Встроенный AI-бот для ответов на вопросы	P1

AI-02	Перевод сообщений (AI)	P2
AI-03	Генерация резюме чата	P2

4. НЕФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

4.1 Производительность

Параметр	Значение	Метод проверки
Одновременные WebSocket соединения	10 000+	Нагрузочное тестирование (k6)
Задержка доставки сообщения	< 100 мс (p95)	Prometheus метрики
Время ответа REST API	< 50 мс	Абсолютно
Время загрузки истории (50 сообщений)	< 200 мс	Индексы MySQL
Потребление RAM на сервере	< 2 GB на 10k пользователей	Мониторинг

4.2 Безопасность

Требование	Реализация
Шифрование трафика	TLS 1.3 (HTTPS + WSS)
Аутентификация	JWT токены (24h expiry, refresh token опционально)
Хэширование паролей	bcrypt (cost=12)
Защита от брутфорса	Rate limiting: 5 попыток/минуту
SQL инъекции	Параметризованные запросы (sqlx)
XSS защита	CSP заголовки, экранирование вывода

4.3 Надежность

Требование	Реализация
Graceful shutdown	Обработка SIGTERM, закрытие соединений
Reconnect стратегия	Exponential backoff на клиенте
Сохранение сообщений	ACID транзакции в MySQL
Health checks	Эндпоинт /health для балансировщика

4.4 Масштабируемость

Компонент	Стратегия масштабирования
WebSocket сервер	Горизонтальное масштабирование через Redis Pub/Sub
База данных	Репликация master-slave → шардирование по chat_id
Файлы	S3-совместимое хранилище (Selectel)

5. СТЕК ТЕХНОЛОГИЙ (Актуальные версии)

5.1 Бэкенд

Компонент	Технология	Версия	Обоснование
Язык	Go	1.22+	Высокая производительность, легковесные горутины, отличная поддержка WebSocket
WebSocket	Gorilla WebSocket	1.5.1	Стандарт де-факто в Go-сообществе
HTTP роутер	chi	5.0.12	Легковесный, совместим со стандартным http.Handler

ORM/Query builder	sqlx	1.3.5	Расширение database/sql, удобная работа со структурами
База данных	MySQL	8.0+	ACID, JSON поддержка, полнотекстовый поиск, репликация
Кеш/брокер	Redis	7.2+	Pub/Sub для горизонтального scaling, хранение сессий
JWT	golang-jwt/jwt	5.2.0	Стабильная библиотека с хорошей документацией
Хэширование	bcrypt	0.0.0	Встроен в x/crypto
Конфигурация	godotenv	1.5.1	12-factor app
Логирование	logrus	1.9.3	Структурированные логи в JSON
Метрики	prometheus/client_golang	1.19.0	Сбор метрик для Grafana

5.2 Клиент

Компонент	Технология	Версия	Обоснование
Фреймворк	Vue 3	3.4+	Composition API, реактивность, простота
State Management	Pinia	2.1+	Официальный store для Vue 3
Сборщик	Vite	5.0+	Мгновенная HMR, быстрая сборка
Язык	TypeScript	5.3+	Типизация, удобство рефакторинга
WebSocket клиент	Native WebSocket API	-	Без лишних зависимостей

HTTP клиент	Axios	1.6+	Удобная работа с перехватчиками
UI компоненты	Naive UI / PrimeVue	-	Легковесные компоненты
Дата/время	dayjs	1.11+	Легкая альтернатива moment.js

5.3 DevOps

Компонент	Технология	Версия
Контейнеризация	Docker	24+
Оркестрация	Docker Compose	2.20+
Веб-сервер	Nginx	1.24+
Мониторинг	Prometheus + Grafana	2.50+ / 10.0+
CI/CD	GitHub Actions	-

6. СТРУКТУРА ПРОЕКТА

chelbo/	# Корень проекта
— backend/	# Go сервер
— cmd/	
— server/	
— main.go	# Точка входа, запуск
сервера, graceful shutdown	
— internal/	# Внутренний код (не
импортируется извне)	
— auth/	# Модуль авторизации
— handler.go	# HTTP обработчики:
Register, Login, Logout	
— middleware.go	# JWT проверка, Rate
limiting, CORS	
— jwt.go	# Генерация и валидация JWT

токенов					
			—	chat/	# Модуль чатов
			—	handler.go	# HTTP: история сообщений,
список чатов					
			—	websocket.go	# WebSocket hub, управление
соединениями					
			—	message.go	# Логика отправки/получения
сообщений					
			—	room.go	# Групповые чаты
			—	user/	# Модуль пользователей
			—	handler.go	# HTTP: профиль, обновление,
поиск					
			—	service.go	# Бизнес-логика
пользователей					
			—	repository.go	# Запросы к БД
			—	file/	# Модуль файлов
			—	handler.go	# Загрузка/скачивание файлов
			—	storage.go	# Локальное или S3 хранилище
			—	image.go	# Ресайз изображений
			—	ai/	# Модуль AI-ассистента
			—	handler.go	# Обработка запросов к AI
			—	assistant.go	# Генерация ответов
(имитация нейросети)					
			—	database/	# Работа с БД
			—	db.go	# Подключение к MySQL, пул
соединений					
			—	migrations/	# Миграции БД
			—	001_create_users.sql	
			—	002_create_chats.sql	
			—	003_create_messages.sql	
			—	004_create_files.sql	
			—	queries/	# Сложные SQL запросы
			—	messages.sql	
			—	users.sql	
			—	redis/	# Работа с Redis
			—	client.go	# Подключение к Redis

				session.go	# Хранение сессий
				pubsub.go	# Pub/Sub для
горизонтального масштабирования					
				models/	# Data Transfer Objects
				user.go	# User, UserResponse,
RegisterRequest...					
				message.go	# Message,
SendMessageRequest					
				chat.go	# Chat, GroupChat
				response.go	# Standard API Response
				pkg/	# Вспомогательные пакеты
				logger/	# Логирование (logrus
обертка)					
				logger.go	
				validator/	# Валидация email, пароля,
телефона					
				validator.go	
				password/	# Хэширование паролей
(bcrypt)					
				password.go	
				config/	# Загрузка конфигурации из
.env					
				config.go	
				web/	# Статические файлы (для
продакшн)					
				index.html	
				.env.example	# Пример переменных
окружения					
				.env	# Переменные окружения (не в
git)					
				go.mod	# Go модуль и зависимости
				go.sum	# Контрольные суммы
зависимостей					
				Dockerfile	# Docker образ для Go
сервера					
				Makefile	# Команды: make run, make
build, make test					

└─ frontend/	# Vue 3 клиент
└─ public/	# Статические файлы
└─ favicon.ico	
└─ src/	
└─ assets/	# Иконки, шрифты, стили
└─ main.css	
└─ logo.svg	
└─ components/	# Vue компоненты
└─ chat/	
└─ ChatList.vue	# Список чатов (левая панель)
└─ ChatWindow.vue	# Окно сообщений (центральная часть)
└─ MessageBubble.vue	# Отдельное сообщение
└─ MessageInput.vue	# Поле ввода с кнопками
└─ TypingIndicator.vue	# Индикатор "печатает..."
└─ auth/	
└─ LoginForm.vue	# Форма входа
└─ RegisterForm.vue	# Форма регистрации
└─ user/	
└─ Profile.vue	# Профиль пользователя
└─ AvatarUpload.vue	# Загрузка аватара
└─ UserSearch.vue	# Поиск пользователей
└─ common/	
└─ Loader.vue	# Спиннер загрузки
└─ Modal.vue	# Модальное окно
└─ Toast.vue	# Уведомления
└─ views/	# Страницы (маршруты)
└─ LoginPage.vue	# Страница входа
└─ RegisterPage.vue	# Страница регистрации
└─ ChatPage.vue	# Основная страница с чатами
└─ SettingsPage.vue	# Настройки
└─ NotFoundPage.vue	# 404
└─ stores/	# Pinia stores (состояние)
└─ authStore.ts	# Токен, текущий

пользователь				chatStore.ts	# Список чатов, сообщения,
активный чат				userStore.ts	# Онлайн статусы, поиск
пользователей				websocketStore.ts	# WebSocket соединение,
обработка событий					
				api/	# API клиенты
				client.ts	# Axios инстанс с
перехватчиками				auth.ts	# Auth API (register, login,
logout)				messages.ts	# Messages API (history,
send)				chats.ts	# Chats API (list, create)
				files.ts	# Files API (upload,
download)					
				composables/	# Vue composables
(переиспользуемая логика)				useWebSocket.ts	# Хук для WebSocket
				useAuth.ts	# Хук для авторизации
				useInfiniteScroll.ts	# Бесконечная подгрузка
истории					
				router/	# Маршрутизация
				index.ts	# Vue Router конфигурация
				types/	# TypeScript типы
				user.ts	
				message.ts	
				chat.ts	
				api.ts	
				utils/	# Утилиты
				date.ts	# Форматирование дат
				storage.ts	# localStorage обертка
				validation.ts	# Валидация форм
				App.vue	# Корневой компонент
				main.ts	# Точка входа (монтируем

```

Vue)
|
|   |— index.html           # HTML шаблон
|   |— package.json         # NPM зависимости
|   |— package-lock.json
|   |— vite.config.ts       # Vite конфигурация
|   |— tsconfig.json        # TypeScript конфигурация
|   |— Dockerfile           # Docker образ для клиента
(Nginx)
|   |— .env.production      # Переменные для продакшн
|
|— docker-compose.yml       # Запуск всех сервисов (Go +
MySQL + Redis + Nginx)
|— docker-compose.dev.yml   # Запуск для разработки
|— nginx.conf               # Nginx конфигурация
(reverse proxy + статика)
|— .github/
|   |— workflows/
|       |— test.yml         # GitHub Actions: запуск
тестов
|       |— deploy.yml       # GitHub Actions: деплой на
VPS
|
|— docs/                   # Документация для диплома
|   |— architecture.md     # Архитектура проекта
|   |— api-spec.yaml       # OpenAPI спецификация
|   |— deployment.md       # Инструкция по
развертыванию
|   |— testing.md          # План тестирования
|   |— diploma-presentation.md # Материалы для защиты
|
|— scripts/
|   |— setup.sh            # Скрипт для первого запуска
|   |— backup-db.sh        # Бэкап базы данных
|   |— load-test.js        # Нагрузочное тестирование
(k6)
|
|— .env.example            # Пример всех переменных
окружения
|— .gitignore              # Что не попадает в git
|— .air.toml               # Конфигурация для live
reload (air)

```

└─ Makefile
командами
└─ README.md
ссылки

Главный Makefile с
Описание проекта, запуск,

7. МОДЕЛИ ДАННЫХ (MySQL 8.0+)

7.1 Таблица users

```
CREATE TABLE users (  
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    email VARCHAR(255) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    avatar_url TEXT,  
    bio VARCHAR(200) DEFAULT '',  
    is_active BOOLEAN DEFAULT TRUE,  
    is_verified BOOLEAN DEFAULT FALSE,  
    last_seen TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
  
    INDEX idx_email (email),  
    INDEX idx_name (name),  
    INDEX idx_last_seen (last_seen)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

7.2 Таблица chats

```
CREATE TABLE chats (  
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    type ENUM('private', 'group') NOT NULL DEFAULT 'private',  
    title VARCHAR(100), -- только для групп  
    avatar_url TEXT, -- только для групп  
    created_by BIGINT UNSIGNED NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
```

```
CURRENT_TIMESTAMP,
```

```
    INDEX idx_type (type),  
    INDEX idx_created_by (created_by),  
    FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

7.3 Таблица chat_participants

```
CREATE TABLE chat_participants (  
    chat_id BIGINT UNSIGNED NOT NULL,  
    user_id BIGINT UNSIGNED NOT NULL,  
    role ENUM('member', 'admin', 'owner') DEFAULT 'member',  
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    last_read_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    PRIMARY KEY (chat_id, user_id),  
    INDEX idx_user_id (user_id),  
    FOREIGN KEY (chat_id) REFERENCES chats(id) ON DELETE CASCADE,  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

7.4 Таблица messages

```
CREATE TABLE messages (  
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    chat_id BIGINT UNSIGNED NOT NULL,  
    sender_id BIGINT UNSIGNED NOT NULL,  
    text TEXT,  
    file_url TEXT,  
    file_type VARCHAR(50),  
    is_deleted BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    INDEX idx_chat_id (chat_id),  
    INDEX idx_sender_id (sender_id),  
    INDEX idx_created_at (created_at),  
    INDEX idx_chat_time (chat_id, created_at DESC), -- для  
пагинации  
    FULLTEXT idx_text (text), -- для поиска
```

```

    FOREIGN KEY (chat_id) REFERENCES chats(id) ON DELETE CASCADE,
    FOREIGN KEY (sender_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

7.5 Таблица `message_statuses`

```

CREATE TABLE message_statuses (
    message_id BIGINT UNSIGNED NOT NULL,
    user_id BIGINT UNSIGNED NOT NULL,
    is_delivered BOOLEAN DEFAULT FALSE,
    is_read BOOLEAN DEFAULT FALSE,
    delivered_at TIMESTAMP NULL,
    read_at TIMESTAMP NULL,

    PRIMARY KEY (message_id, user_id),
    INDEX idx_user_unread (user_id, is_read),
    FOREIGN KEY (message_id) REFERENCES messages(id) ON DELETE
CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

8. API ЭНДПОИНТЫ (REST + WebSocket)

8.1 REST API

Метод	Путь	Описание	Auth
POST	/api/auth/register	Регистрация	✗
POST	/api/auth/login	Вход	✗
POST	/api/auth/logout	Выход	✓
GET	/api/auth/me	Текущий пользователь	✓

PUT	/api/users/me	Обновить профиль	✓
POST	/api/users/me/avatar	Загрузить аватар	✓
GET	/api/users/search?q=...	Поиск пользователей	✓
GET	/api/chats	Список чатов	✓
POST	/api/chats/private/:userId	Создать личный чат	✓
POST	/api/chats/group	Создать группу	✓
GET	/api/chats/:chatId/messages?limit=50&offset=0	История	✓
POST	/api/chats/:chatId/messages	Отправить сообщение (через REST)	✓
DELETE	/api/messages/:messageId	Удалить сообщение	✓
POST	/api/files/upload	Загрузить файл	✓
GET	/api/files/:fileId	Скачать файл	✓
POST	/api/ai/ask	Запрос к AI	✓
GET	/health	Health check	✗
GET	/metrics	Метрики Prometheus	✗

8.2 WebSocket (WSS)

Подключение: `wss://api.chelbo.ru/ws?token={jwt}`

Сообщения от клиента:

```
// Отправить сообщение
{
  "type": "message",
  "chat_id": 123,
  "text": "Привет!"
}
```

```
// Печатает...
{
  "type": "typing",
  "chat_id": 123,
  "is_typing": true
}
```

```
// Прочитано
{
  "type": "read",
  "chat_id": 123,
  "message_id": 456
}
```

Сообщения от сервера:

```
// Новое сообщение
{
  "type": "new_message",
  "message": {
    "id": 456,
    "chat_id": 123,
    "sender_id": 1,
    "text": "Привет!",
    "created_at": "2024-01-15T10:30:00Z"
  }
}
```

```
// Пользователь печатает
```

```
{
  "type": "typing",
  "chat_id": 123,
  "user_id": 2,
  "is_typing": true
}

// Пользователь онлайн
{
  "type": "user_online",
  "user_id": 2
}

// Подтверждение доставки
{
  "type": "message_delivered",
  "message_id": 456,
  "chat_id": 123
}
```

9. ЗАПУСК ПРОЕКТА (Инструкция)

9.1 Требования

- Go 1.22+
- MySQL 8.0+
- Redis 7.2+
- Node.js 20+ (для фронтенда)
- Docker (опционально)

9.2 Быстрый старт с Docker (рекомендуется)

```
# Клонировать репозиторий
git clone https://github.com/your/chelbo.git
cd chelbo
```

```
# Скопировать переменные окружения
cp .env.example .env
```

```
# Запустить все сервисы
docker-compose up -d

# Применить миграции
docker exec -it chelbo-backend go run cmd/migrate/main.go

# Открыть в браузере
open http://localhost
```

9.3 Ручной запуск

```
# Бэкенд
cd backend
go mod download
go run cmd/server/main.go

# Фронтенд
cd frontend
npm install
npm run dev

# MySQL
sudo systemctl start mysql
mysql -u root -p <
backend/internal/database/migrations/001_create_users.sql

# Redis
redis-server
```

9.4 Makefile команды

make help	# Показать все команды
make dev	# Запустить в режиме разработки (live reload)
make build	# Собрать бинарник
make test	# Запустить тесты
make docker-up	# Запустить Docker контейнеры
make docker-down	# Остановить Docker
make migrate-up	# Применить миграции
make migrate-down	# Откатить миграции

```
make lint          # Проверить код
```

10. ПЕРЕМЕННЫЕ ОКРУЖЕНИЯ (.env)

```
# Сервер
SERVER_PORT=8080
SERVER_HOST=0.0.0.0
ENVIRONMENT=development # development, production
```

```
# База данных (MySQL)
DB_HOST=localhost
DB_PORT=3306
DB_USER=chelbo
DB_PASSWORD=secure_password
DB_NAME=chelbo
DB_MAX_OPEN_CONNS=100
DB_MAX_IDLE_CONNS=10
```

```
# Redis
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
REDIS_DB=0
```

```
# JWT
JWT_SECRET=your-super-secret-key-change-me
JWT_EXPIRY_HOURS=24
```

```
# Файлы (локальное хранилище или S3)
STORAGE_TYPE=local # local, s3
STORAGE_PATH=./uploads
MAX_FILE_SIZE_MB=50
```

```
# AI (имитация)
AI_ENABLED=true
AI MOCK_RESPONSES=true
```

```
# Rate limiting
RATE_LIMIT_REQUESTS=100
```

```
RATE_LIMIT_WINDOW_MINUTES=1
```

```
# CORS
```

```
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://localhost
```

```
# Логирование
```

```
LOG_LEVEL=debug # debug, info, warn, error
```

```
LOG_FORMAT=text # text, json
```

11. КЛЮЧЕВЫЕ ОСОБЕННОСТИ ДЛЯ ДИПЛОМА

11.1 Чем Chelbo отличается от Telegram

Аспект	Telegram	Chelbo
Клиент	Нативный (C++)	Web + PWA (Vue 3)
Сервер	Собственный (C++)	Go + MySQL
Протокол	MTProto	WebSocket + REST
Установка	Требуется	Не требуется
Адаптация под РФ	✗	✓

11.2 Инновации в проекте

1. **Web-first подход** — мессенджер работает сразу в браузере
2. **Горизонтальное масштабирование** через Redis Pub/Sub
3. **Встроенный AI-ассистент** (имитация нейросети)
4. **PWA** — установка на телефон без App Store

11.3 Демонстрация на защите

1. Открыть Chelbo в браузере (без установки)
2. Зарегистрироваться за 30 секунд
3. Отправить сообщение в real-time
4. Показать историю с пагинацией
5. Показать AI-ассистента

6. Показать графики метрик (Prometheus + Grafana)

12. РИСКИ И ИХ МИТИГАЦИЯ

Риск	Вероятность	Митигация
Высокая нагрузка на WebSocket	Средняя	Горизонтальное масштабирование, Redis Pub/Sub
Медленные запросы в MySQL	Низкая	Индексы, партиционирование по времени
Потеря сообщений	Низкая	ACID транзакции, подтверждения доставки
DDoS атаки	Низкая	Rate limiting, Nginx лимиты
Утечка JWT	Средняя	HTTPS, короткий TTL, refresh токены

13. ДАЛЬНЕЙШЕЕ РАЗВИТИЕ (Roadmap)

Версия	Срок	Фичи
v1.0	День защиты	MVP: регистрация, личные чаты, WebSocket
v1.1	+1 месяц	Группы, файлы, PWA
v1.2	+2 месяца	Аудиозвонки (WebRTC), шифрование
v2.0	+6 месяцев	Мобильные приложения (Flutter)

14. ПОДДЕРЖКА АВТОРА

Проект полностью открыт для вопросов. Все файлы будут предоставлены с комментариями на русском языке.

Контакты для комиссии: (твои данные)

Версия документа: 1.0

Дата: Апрель 2026

Статус: Утвержден к реализации

Этот документ полностью готов для:

- Защиты диплома (показывает инженерный подход)
- Передачи разработчику (всё описано до мелочей)
- Презентации инвестору (есть бизнес-метрики)

Что дальше? Я могу:

1. Написать полный код бэкенда (main.go + каждый модуль)
2. Сгенерировать миграции для MySQL
3. Написать фронтенд на Vue 3
4. Создать docker-compose.yml для быстрого развертывания

Выбери, с чего продолжим!